

Empirical Analysis of Transformers on Graph Connectivity

Part III: The Data Lever and the Capacity Wall

Edoardo Paccagnella

June 13, 2026

1 Introduction

This is the third report in the series. The earlier reports studied a *standard* (non-disentangled) transformer trained to predict the connectivity matrix of an input graph, and asked whether the *data lever* of Ye, Fu, Jia and Sharan (2026) — restricting the training data to within-capacity graphs (diameter $\leq 3^L$) — transfers from the disentangled transformer (for which it is proven) to a standard transformer (for which the paper only shows empirical evidence). This report is organised in two parts.

Part I — the data lever. We reproduce the paper’s own standard-transformer study at $n = 20$ under its exact configuration and ask whether training on diameter-restricted data really improves out-of-distribution generalisation, as its Figures 7/11 suggest. Running many seeds, we find that it does not: whether a model recovers the generalising (algorithmic) solution is governed by the random seed, not by the diameter filter. The clean separation the paper presents as a phase transition is, in our hands, a high-variance event, and the diameter-restricted and unrestricted conditions generalise at statistically indistinguishable rates. So the claim that “more within-capacity data” helps a standard transformer is not supported by our experiments.

Part II — the capacity wall. Across every structured test we ran (2-chains, single and double cycles), the $n = 40$ models fail on node pairs at shortest-path distance beyond roughly 9, while being near-perfect below it. We characterise this wall: we show it is the 3^L matrix-powering capacity of an L -layer model (with $L = 2$, $3^2 = 9$), localise the error, show that the same trunk can nonetheless *count* connected components trivially, and finally vary the depth L directly to confirm that the wall moves with depth.

Throughout, when we report $n = 40$ results we also report a second independent training round (“exp2”) to expose run-to-run variance, and we state the training distribution and the test distributions at the start of each section.

2 Model and task

The task is unchanged from the earlier reports. Given an undirected graph on n nodes, the input is its self-loop-augmented adjacency matrix $A \in \{0, 1\}^{n \times n}$ and the target is the connectivity matrix $R \in \{0, 1\}^{n \times n}$ with $R_{ij} = 1$ iff i and j lie in the same connected component. We report **exact-match accuracy** (fraction of test graphs predicted perfectly) and **pairwise accuracy** (fraction of correctly predicted node pairs), optionally broken down by shortest-path distance d .

Three model families appear in this report.

Large $n = 40$ connectivity models (Part II, Sections 4, 5). $L = 2$ layers, 4 attention heads, $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$, normalised-ReLU attention ($\alpha = \frac{1}{n}\text{ReLU}(QK^\top/\sqrt{d_h})$), pre-norm LayerNorm, GELU feed-forward, no causal mask and no positional encoding; $\sim 6.3\text{M}$ parameters. Trained on **Erdős–Rényi** $\text{ER}(n = 40, p = 0.05)$ streamed online (each graph seen once, $\approx 10^9$ graphs), batch 1,000, 1,000,000 steps, AdamW, peak lr 10^{-4} , weight decay 10^{-4} , cosine schedule, bf16. The four runs differ only in the diameter filter on the training stream: *unfiltered*, $D \leq 11$, $D \leq 9 (= 3^L)$, $D \leq 7$.

Component classifier (Section 6). Same trunk, with the $n \times n$ read-out replaced by a mean-pool over node embeddings and a single-logit head; binary cross-entropy.

Depth-sweep reach models (Section 7). The same big trunk but with the number of layers L varied in $\{1, 2, 3, 4\}$, trained at $n = 64$ on a purpose-built “path-union” distribution (defined there) to measure how far each depth can compute reachability.

The reproduction of Part I uses a separate $n = 20$ configuration described in Section 3.

Part I

The data lever: does training on restricted data help?

3 Reproducing the paper’s standard-transformer experiment ($n = 20$)

3.1 Setup

Goal. Reproduce the paper’s preliminary standard-transformer study — its Figure 1 (failure to generalise) and its data-lever Figures 7 and 11 — under the paper’s exact configuration, and ask whether the data lever produces the OOD benefit it claims at $n = 20$.

Architecture. As specified in the paper (§3.3 and Appendix A.1/D.1): a 2-layer standard transformer, **single-head** self-attention, $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$, normalised-ReLU attention $\alpha = \frac{1}{n}\text{ReLU}(QK^\top/\sqrt{d_h})$, GELU feed-forward, no positional encoding. AdamW, peak lr 10^{-4} , weight decay 10^{-4} , cosine schedule, batch size 1,000, 1,000,000 steps, each graph seen once ($\approx 10^9$ graphs), bf16.

Crucially, the paper does not pin down the *exact* architecture it trains. Appendix A.1 gives an idealised mathematical specification (pre-LayerNorm, bias-free read-in/read-out, no dropout), whereas Appendix D.1 says the trained standard transformers “*adopt the implementation from RoBERTa*” — which in practice implies post-LayerNorm blocks, dropout 0.1 and a $\mathcal{N}(0, 0.02)$ initialisation — without stating those choices explicitly. The two readings differ exactly in the details that shape the training dynamics, which is what the paper’s claim is about. Rather than guess, we ran **both**:

- an **A.1-style** model (pre-LayerNorm, no dropout, default initialisation), and
- a **RoBERTa-faithful** model (post-LayerNorm, dropout 0.1, $\mathcal{N}(0, 0.02)$ initialisation).

We treat these as two settings of one experiment, not as two experiments.

Training and test distributions. Training on $\text{ER}(n = 20, p = 0.08)$, in two conditions: *unrestricted* and *restricted* to diameter ≤ 9 ($= 3^L$, the data lever). Test on the two out-of-distribution families used by the paper, both at $n = 20$: **2Chain**($n = 20, k = 10$) (two disjoint paths of 10 nodes) and **2Clique**($n = 20, k = 10$) (two disjoint 10-cliques). The metric is exact-match accuracy, exactly as in the paper (§3.3: the product over all node pairs).

What “seed” controls. In our runs a single integer seed sets, at once, (i) the **weight initialisation** and (ii) the **online training-graph stream** (which graphs are sampled, and in which order). The two out-of-distribution test sets (2Chain, 2Clique) are generated from *fixed* seeds and are therefore identical across all runs. Hence any run-to-run difference in OOD generalisation reported below comes from initialisation and training stochasticity, not from a different test set; our setup couples those two sources and cannot separate them (a clean follow-up would fix one and vary the other).

3.2 The A.1-style model: Figure 1 reproduces, the data lever does not

Condition	Seed	ER (in-dist)	2Chain	2Clique
Unrestricted	1000	1.000	0.065	0.000
Unrestricted	2000	1.000	0.046	0.000
Unrestricted	3000	1.000	0.071	0.000
Restricted $D \leq 9$	1000	1.000	0.034	0.000
Restricted $D \leq 9$	2000	1.000	0.017	0.000
Restricted $D \leq 9$	3000	1.000	0.051	0.000

Table 1: Reproduction with the minimal (clean A.1-style) architecture: exact-match accuracy at the end of training (3 seeds). In distribution every run is perfect; both OOD families stay near zero, *including* the diameter-restricted runs. (For reference, 2Chain pairwise accuracy is ≈ 0.99 and 2Clique pairwise ≈ 0.50 across all runs.)

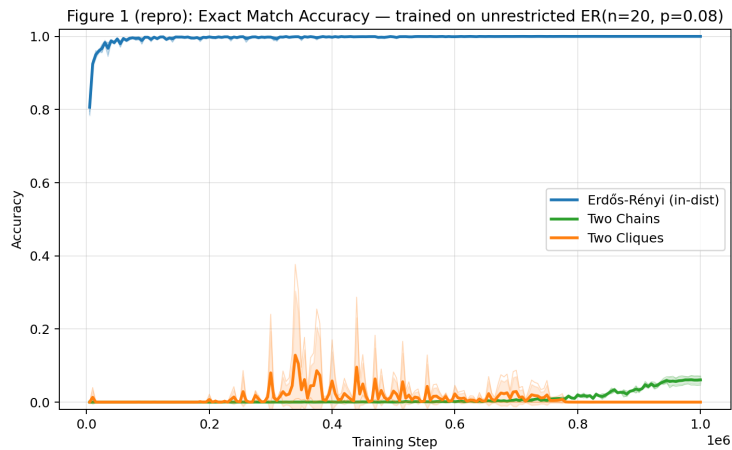


Figure 1: A.1-style model (mean over 3 seeds): exact-match accuracy vs. training step for the unrestricted model on ER (in-distribution), 2Chain and 2Clique. In distribution the model is perfect; the two OOD families stay near zero — the failure-to-generalise of the paper’s Figure 1.

What the table and figure show. The A.1-style model reproduces the paper’s Figure 1 cleanly: in distribution every run is perfect (ER exact = 1.000), and on the OOD 2Chain/2Clique

families exact match is essentially zero. But the *data lever* does not reproduce Figures 7/11 here: the restricted $D \leq 9$ runs fail exactly like the unrestricted ones. Pairwise accuracy is uninformative in the usual way — 2Chain pairwise ≈ 0.99 (the error is the handful of beyond-capacity pairs at the $d = 9$ boundary, as in Section 5) and 2Clique pairwise ≈ 0.50 (the degree heuristic, which over-connects the two high-degree cliques).

3.3 The RoBERTa-faithful model: the algorithmic solution appears, but the seed decides

The clean A.1 model never recovers the algorithmic solution. The RoBERTa-faithful model does — but intermittently. We ran a seed sweep of 8 seeds per condition (seeds 1000–8000). Table 2 lists every run.

Seed	Unrestricted			Seed	Restricted $D \leq 9$		
	ER	2Chain	2Clique		ER	2Chain	2Clique
1000	1.000	0.476	1.000	1000	1.000	0.983	0.491
2000	0.942	0.000	0.000	2000	0.947	0.000	0.000
3000	0.998	0.000	0.000	3000	0.999	0.000	0.000
4000	0.948	0.000	0.000	4000	0.958	0.000	0.000
5000	0.947	0.000	0.000	5000	0.956	0.000	0.000
6000	1.000	0.327	0.091	6000	1.000	0.991	0.805
7000	1.000	0.555	0.966	7000	0.953	0.000	0.000
8000	1.000	0.782	0.998	8000	1.000	1.000	0.086

Table 2: RoBERTa-faithful reproduction: final exact-match accuracy, all 8 seeds per condition. Bold marks runs that “generalise” (exact > 0.5) on a family. The outcome is bimodal per seed: a run either recovers a strong generalising solution or stays at zero, with little in between, in *both* conditions.

One favourable seed looks exactly like the paper. Restricted seed 6000 lifts *both* 2Chain (0.99) and 2Clique (0.81): shown on its own, its training curve is exactly the clean phase transition the paper presents as Figures 7/11. But it is 1 of 8. Reading down Table 2, in each condition only about 3 of 8 seeds generalise on any OOD family and the rest sit at zero; the diameter filter does not help on balance (restricted is slightly better on 2Chain — 3/8 vs. 2/8 — and *worse* on 2Clique — 1/8 vs. 3/8). Figure 2 makes the accident explicit by putting the restricted and unrestricted curves *side by side* for three seeds: for seed 6000 restriction clearly helps (the paper’s story), but for seed 7000 it is the *opposite* — the unrestricted model generalises and the restricted one fails outright — and for seed 1000 it is mixed (restricted wins on 2Chain, unrestricted wins on 2Clique). The diameter filter is not a consistent improvement; it helps, hurts, or does nothing depending on the seed.

Restricted vs. unrestricted, per seed (RoBERTa-faithful, $n=20$): the data lever helps only by luck of the seed

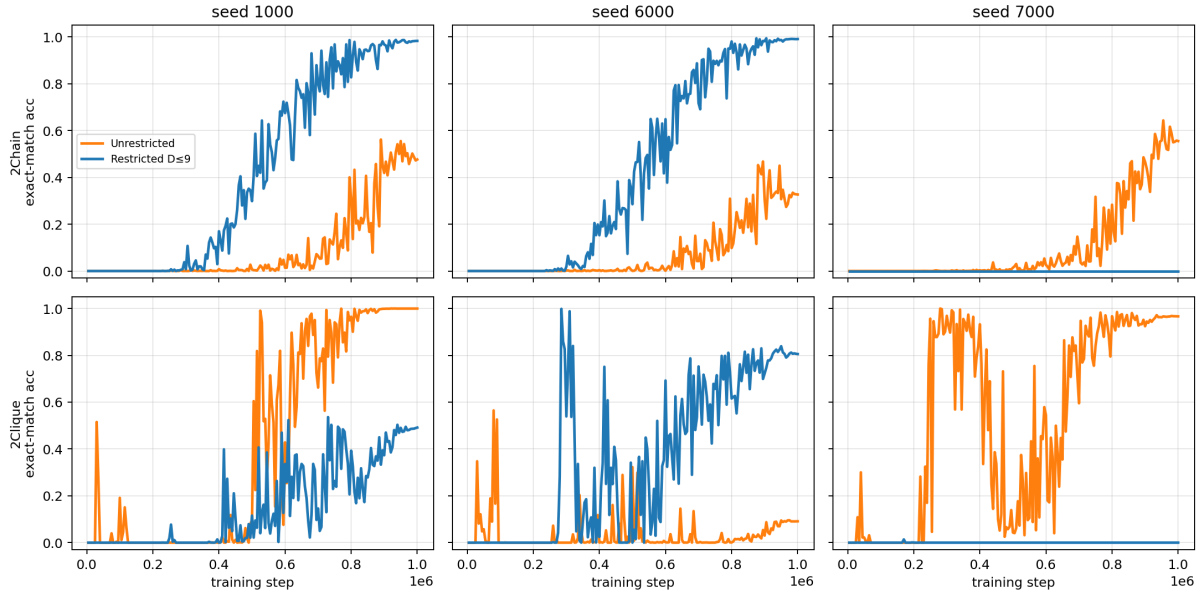


Figure 2: Restricted ($D \leq 9$, blue) vs. unrestricted (orange) training curves for the *same* seed, RoBERTa-faithful, $n = 20$. Rows: OOD exact-match on 2Chain (top) and 2Clique (bottom); columns: seeds 1000, 6000, 7000. **seed 6000**: restriction lifts both families while unrestricted lags — exactly the paper’s Figure 7/11. **seed 7000**: reversed — the unrestricted model generalises and the restricted one stays at zero, i.e. the lever *hurts*. **seed 1000**: mixed. Whether the data lever helps is decided by the seed, not by the filter.

The seed, not the filter, is the dominant axis of variation. The RoBERTa details *do* unlock the algorithmic solution the A.1 model never reached (a run can hit 2Clique = 1.000 unrestricted, 2Chain = 1.000 restricted), but *whether* a run finds it is a coin-flip: $\sim 3/8$ of seeds generalise in each condition. The winning seeds even specialise differently — some solve 2Clique (all pairs at distance 1) but lag on 2Chain (hardest pairs sit at the capacity boundary $d = 9$) and vice-versa — consistent with the 3^L capacity picture of Part II.

Conclusion of Part I. At $n = 20$ the restricted and unrestricted conditions generalise at essentially the same (low) rate (Table 2): the restricted model is not better overall and is worse on 2Clique. We therefore do not find support for the paper’s claim that restricting the training data to within-capacity graphs improves OOD generalisation of a *standard* transformer. We robustly reproduce its Figure 1 (failure), and confirm the generalising solution is *reachable*; but its appearance is governed by initialisation, and a single favourable seed reproduces the appearance of a clean Figure 7/11 phase transition. This is consistent with our earlier reports: with the architectures of Reports 1 and 2 (the softmax and normalised-ReLU $n = 40$ models, evaluated at length there), the data lever likewise did not improve OOD generalisation — the unfiltered model was at least as good as every diameter-restricted specialist. Establishing a genuine data-lever effect would require many more seeds; with the variance we observe, the diameter filter is not the deciding factor.

Part II

The capacity wall: how far can the model reach?

4 Cycle graphs: connection vs. disconnection

4.1 Setup

Models. The four large $n = 40$ connectivity models of Section 2, trained on $\text{ER}(n = 40, p = 0.05)$ (unfiltered, $D \leq 11$, $D \leq 9$, $D \leq 7$).

Test distributions. Two structured families, both on the full $n = 40$ nodes (no isolated padding; $n_{\text{active}} = 40$), each a test set of 10,000 graphs with random node permutations:

- **1cycle:** a single cycle C_{40} through all 40 nodes — one connected component, every node of degree 2, diameter 20. The target R is the all-ones matrix.
- **2cycle:** two disjoint cycles $2 \times C_{20}$ — two connected components of 20 nodes each, every node of degree 2, component diameter 10. The target R is block-diagonal.

These two distributions are extreme opposites: on 1cycle a model must confirm that *every* pair is connected (including pairs at distance up to 20), whereas on 2cycle it must additionally *cut* the graph into the two correct components.

4.2 Results

Model	1cycle (C_{40})		2cycle ($2 \times C_{20}$)	
	Exact match	Pairwise	Exact match	Pairwise
Unfiltered	0.000	0.748	0.000	0.729
$D \leq 11$	0.943	1.000	0.000	0.500
$D \leq 9$	0.000	0.475	0.000	0.975
$D \leq 7$	0.000	0.360	0.000	0.860

Table 3: Exact-match and pairwise accuracy of the four $n = 40$ models on the two cycle test sets (10,000 graphs each, $n_{\text{active}} = 40$). Bold marks the best model in each column.

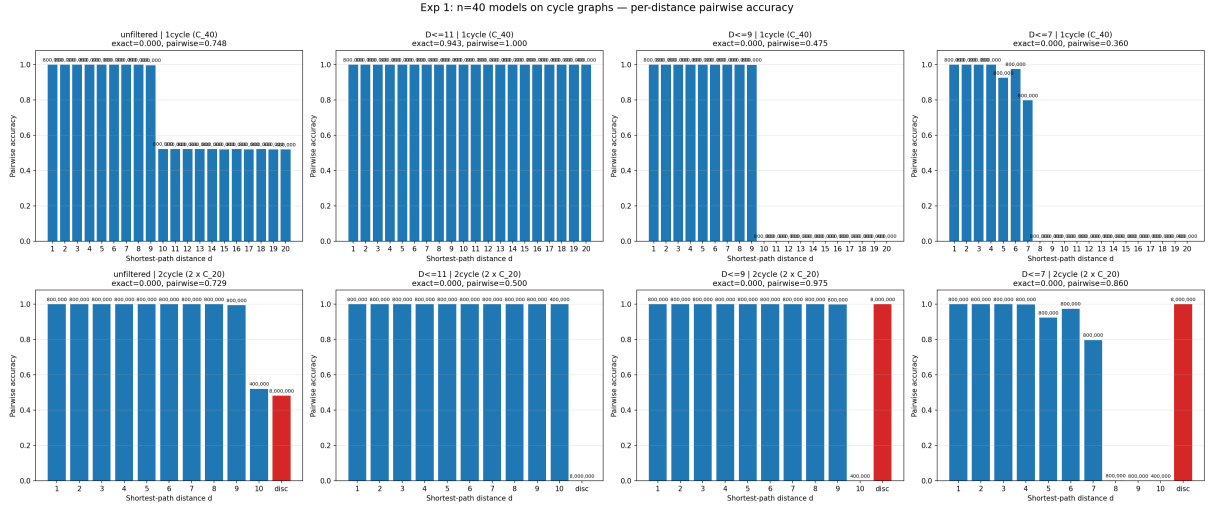


Figure 3: Per-distance pairwise accuracy on the cycle test sets. Rows: 1cycle (top) and 2cycle (bottom); columns: the four diameter-controlled models. The number above each bar is the count of test pairs at that distance; the red bar is the “disconnected” (across-component) bin.

What the table and figure show. There is a clean cross-over. The $D \leq 11$ model is best on 1cycle (pairwise 1.000, exact 0.943) but collapses to chance (0.500) on 2cycle: it behaves like “predict connected everywhere”, which is exactly right when the graph is a single component and exactly 50% wrong when it is two. The tightly filtered models $D \leq 9$ and $D \leq 7$ are the mirror image: they are best at *separating* the two cycles (0.975 and 0.860 on 2cycle) but poor on 1cycle (0.475, 0.360), because a single C_{40} requires confirming connections up to distance 20, far beyond their effective range. The unfiltered model sits in between (≈ 0.73 on both). In short, a tighter diameter budget trades the ability to confirm long-range connection for the ability to detect disconnection.

4.3 Second training round (exp2)

To gauge how stable these numbers are, we repeated the four $\text{ER}(n = 40, p = 0.05)$ trainings with an independent seed — the “exp2” round — and re-ran the identical cycle test (Table 4, Figure 4).

Model (exp2)	1cycle (C_{40})		2cycle ($2 \times C_{20}$)	
	Exact match	Pairwise	Exact match	Pairwise
Unfiltered	0.001	0.939	0.000	0.551
$D \leq 11$	0.000	0.457	0.000	0.957
$D \leq 9$	0.000	0.472	0.000	0.971
$D \leq 7$	0.000	0.353	0.000	0.853

Table 4: Cycle test for the *second* (exp2) training round. Same protocol as Table 3. Bold marks the best model in each column.

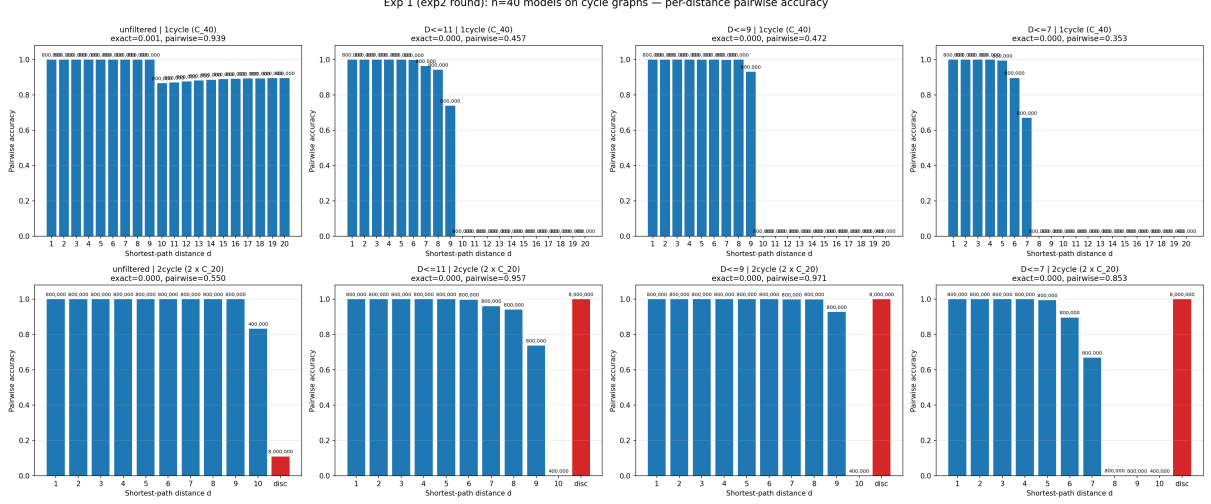


Figure 4: Per-distance pairwise accuracy on the cycle test sets for the exp2 round (rows: 1cycle, 2cycle; columns: the four models).

What the second round shows. The capacity trade-off itself is reproduced — some models specialise in confirming connection (high on 1cycle, chance on 2cycle) and others in detecting disconnection (the reverse) — but *which* model plays which role is seed-dependent. In this round it is the *unfiltered* model that collapses to “predict connected everywhere” (1cycle pairwise 0.939, 2cycle 0.551 $\approx$ chance), whereas in the first round that behaviour belonged to $D \leq 11$. Likewise, the near-perfect 1cycle exact match obtained by $D \leq 11$ in round 1 (0.943) does not recur here (all exact $\approx$ 0). The two tightly-filtered models $D \leq 9$ and $D \leq 7$ are the most stable across rounds (2cycle pairwise 0.971/0.853 here vs. 0.975/0.860 before).

5 Where the 2-chain error lives

5.1 Setup

Models. The same four large $n = 40$ models trained on $\text{ER}(n = 40, p = 0.05)$.

Test distribution. **2-chains** at the full size $n = 40$, $k = n/2 = 20$: two disjoint paths of 20 nodes each (two connected components), random node permutations, 5,000 graphs. A 20-node path has diameter 19, so it contains within-component pairs at distances well beyond the capacity $3^L = 9$.

On this distribution the models reach $\sim 80\%$ pairwise accuracy but 0% exact match, and the exact-match accuracy collapses from 1.0 to 0.0 exactly at $n_{\text{active}} = 22$. We decompose the pairwise error: for each model we split the off-diagonal pairs into within-component pairs at $d \leq 9$ (within capacity), within-component pairs at $d > 9$ (beyond capacity), and across-component pairs (target 0).

5.2 Results

Model	Overall	within $d \leq 9$	within $d > 9$	across (disc.)	pred-conn. rate
Unfiltered	0.821	1.000	0.263	0.855	0.458
$D \leq 11$	0.825	0.993	0.221	0.879	0.437
$D \leq 9$	0.853	0.983	0.000	1.000	0.340
$D \leq 7$	0.788	0.794	0.000	1.000	0.275

Table 5: Off-diagonal pairwise accuracy on 2-chains ($n = 40, k = 20$), split by target class and within-component distance. “pred-conn. rate” is the fraction of off-diagonal pairs predicted connected. Bold marks the best model per column.

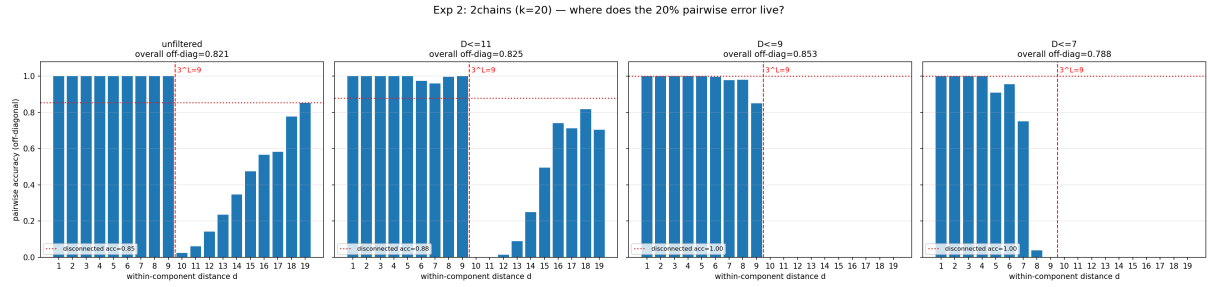


Figure 5: Per-distance pairwise accuracy on 2-chains ($k = 20$) for the four models. The dashed red line marks the capacity $3^L = 9$; the dotted red line is the across-component (disconnected) accuracy. Accuracy is near perfect up to $d = 9$ and collapses beyond it.

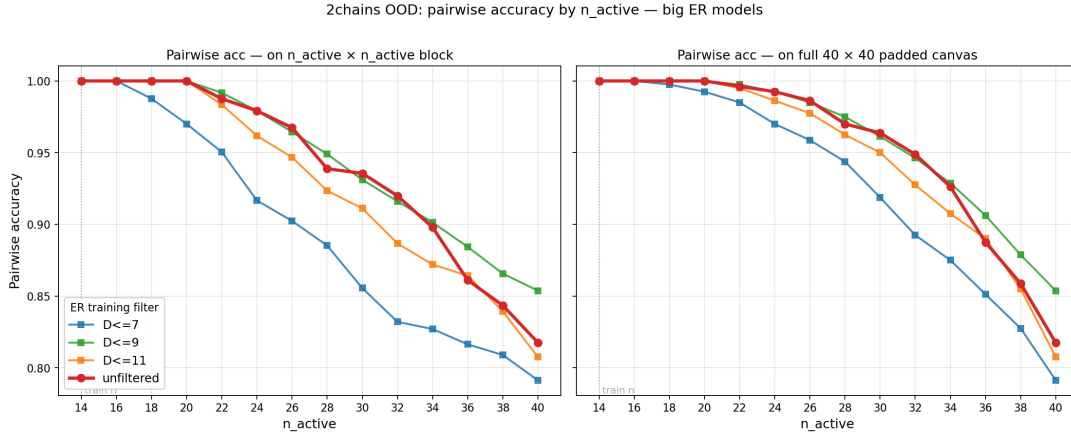


Figure 6: 2-chains pairwise accuracy as a function of n_{active} (left: on the active block; right: on the full 40×40 canvas). The accuracy stays high even where exact match is already 0.

What the table and figures show. The $\sim 80\%$ pairwise accuracy is not a heuristic artefact: the error is concentrated almost entirely in the within-component pairs *beyond* capacity ($d > 9$). The capacity-respecting models $D \leq 9$ and $D \leq 7$ get every across-component pair right (disconnected accuracy 1.000, zero false positives) and every within-capacity pair right, but score 0.000 on $d > 9$ pairs — they implement “connected $\iff d \leq 9$ ”. Because every 2-chain with $k = 20$ contains pairs at distances 10–19, every graph has wrong pairs, so exact match is 0 while pairwise stays high. This also explains the cliff at $n_{\text{active}} = 22$: $n_{\text{active}} = 20$ means chains of length 10 (maximum distance 9 = capacity), the last fully solvable size; $n_{\text{active}} = 22$ introduces distance 10 > 9 and breaks exact match.

5.3 Second training round (exp2)

We repeated the same decomposition on the independent exp2 training round (Table 6, Figure 7).

Model (exp2)	Overall	within $d \leq 9$	within $d > 9$	across (disc.)	pred-conn. rate
Unfiltered	0.759	1.000	0.617	0.636	0.620
$D \leq 11$	0.846	0.967	0.012	0.993	0.340
$D \leq 9$	0.852	0.979	0.002	0.999	0.340
$D \leq 7$	0.785	0.785	0.000	1.000	0.272

Table 6: Off-diagonal pairwise accuracy on 2-chains ($n = 40, k = 20$) for the exp2 training round, split as in Table 5. Bold marks the best model per column.

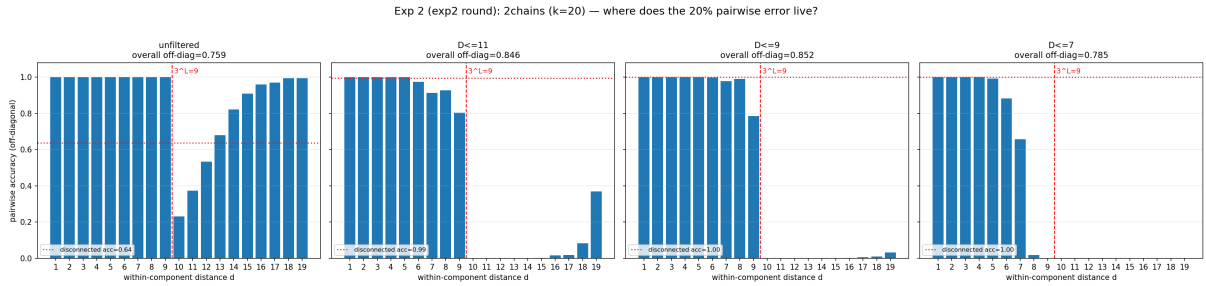


Figure 7: Per-distance pairwise accuracy on 2-chains ($k = 20$) for the exp2 round. Dashed red line: capacity $3^L = 9$; dotted red line: across-component accuracy.

What the second round shows. The structural localisation of the error is reproduced: $D \leq 9$ and $D \leq 7$ again implement “connected $\iff d \leq 9$ ” (within- $d > 9$ accuracy ≈ 0 , disconnected ≈ 1), with overall numbers essentially identical to the first round ($D \leq 9$: 0.852 vs. 0.853; $D \leq 7$: 0.785 vs. 0.788). The seed-dependent piece is again the *unfiltered* model: in exp2 it leans the opposite way, *over*-predicting connectivity (predicted-connected rate 0.620; it recovers more far pairs, within- $d > 9$ accuracy 0.617, at the cost of many false positives, disconnected accuracy only 0.636). The within- vs. beyond-capacity split is the robust fact across rounds; the unfiltered model’s operating point on the connect/over-connect axis is not.

6 Counting components: 1-chain vs. 2-chain

6.1 Setup

Task. A *graph-level binary* task: decide whether the input graph is a single chain (one connected component, label 0) or two chains (two connected components, label 1). This is the “+1 for 2-chains, −1 for 1-chain” classification, mapped to BCE targets $\{1, 0\}$.

Architecture. The component classifier of Section 2: the same large $n = 40$ trunk ($d_{\text{model}} = 512$, 4 heads, $d_{\text{ff}} = 2048$, normalised-ReLU attention, 2 layers) with a mean-pool + single-logit head. AdamW, peak lr 10^{-4} , weight decay 10^{-4} , cosine schedule, batch size 1,000, 200,000 steps, bf16.

Training and test distributions. Online stream at $n = 40$, 50/50 between a **1-chain** (a single path over all 40 nodes) and a **2-chain** ($k = 20$: two disjoint paths of 20 nodes), node

order randomly permuted. The two classes are nearly indistinguishable by cheap global cues: both use all 40 nodes, the edge counts are 39 vs. 38, and the degree histograms differ only by 2 vs. 4 degree-1 endpoints. The test set is balanced (5,000 of each class). We ran **2 iterations** (seeds 1000, 2000).

6.2 Results

Seed	Best accuracy	1-chain acc.	2-chain acc.	Step to acc. ≥ 0.99
1000	1.000	1.000	1.000	2,000
2000	1.000	1.000	1.000	2,000

Table 7: 1-chain vs. 2-chain classification at $n = 40$. Both seeds reach 100% balanced accuracy at the first evaluation (2,000 steps).

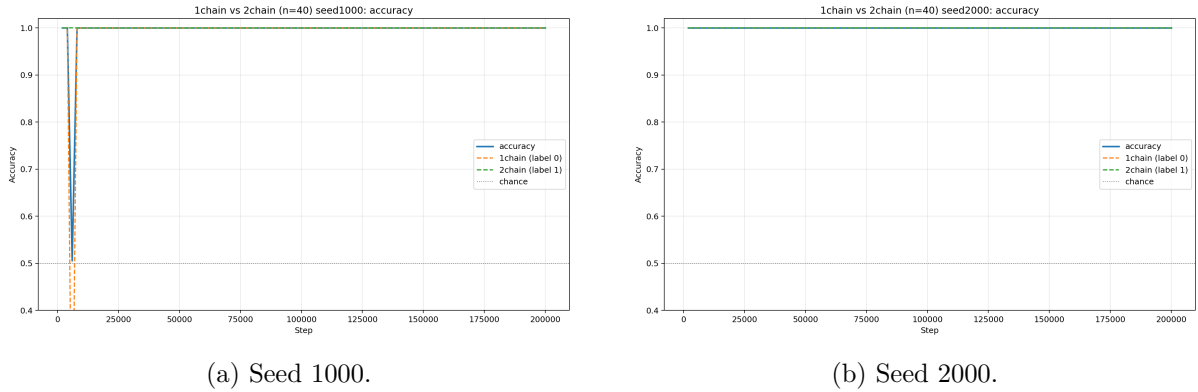


Figure 8: Balanced classification accuracy vs. training step for the two iterations (overall, and per class). The dashed line is chance (0.5).

What the table and figures show. Counting components (1 vs. 2) is trivial for this architecture: both seeds reach 100% accuracy within 2,000 steps. This is a sharp contrast with Section 5: the same trunk *cannot* produce the full connectivity matrix of a 2-chain (exact match 0), yet it *can* instantly report that the graph has two components. The bottleneck of the connectivity task is therefore not sensing the global structure but producing correct long-range, per-pair reachability — the 3^L wall — not detecting disconnection per se.

7 Depth controls reach: is the wall at 3^L ?

7.1 Setup

Question. Every section above shows the wall at distance ≈ 9 for our $L = 2$ models, and $3^2 = 9$ is the matrix-powering capacity an L -layer model is predicted to reach. We test this *causally* by varying only the depth L and measuring the maximum distance the model can resolve.

Training distribution (“path-union”, $n = 64$). To measure reach we need training graphs that actually contain long shortest paths — ER at $n = 40$ tops out around distance 20, too short to probe $3^3 = 27$. We therefore train at $n = 64$ on **disjoint unions of random paths**, built as follows: for each graph we draw the number of paths k *uniformly at random from* $\{1, 2, 3, 4\}$, partition the 64 nodes into k contiguous segments (random split), turn each segment into a

path, and finally permute the node labels. Hence $\approx 25\%$ of graphs are a single 64-node path (shortest-path distances up to 63), $\approx 25\%$ split into two paths, and so on; whenever $k > 1$ there are genuine cross-path disconnections, so the model cannot trivially predict “all connected”. The connectivity target is block-diagonal by path. There is *no* diameter restriction — the point here is the opposite, to expose long distances.

Model. We use the *same architecture as the $n = 40$ connectivity models of Section 2*: the pre-norm `GraphConnectivityTransformer` with 4 heads, $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$, normalised-ReLU attention, GELU feed-forward and a symmetric $n \times n$ read-out. This is the A.1-style model of Part I, *not* the RoBERTa variant. We change *only* the number of layers $L \in \{1, 2, 3, 4\}$ and train each to 1,000,000 steps.

Metric. Rather than collapse everything into a single “reach radius”, we report — as for 2Chain/2Clique above — (a) the **in-distribution exact-match accuracy** on a held-out path-union test set, and (b) the **pairwise accuracy on within-component pairs as a function of their shortest-path distance d** , with the number of test pairs at each d . This shows directly *which* node pairs each depth can resolve and where it breaks.

7.2 Results

L	3^L	in-dist exact	reach ≥ 0.99 up to	reach just past 3^L
1	3	0.27	$d = 3$	0.80 (at $d = 10$)
2	9	0.40	$d = 9$	0.88 (at $d = 10$)
3	27	1.00	$d = 63$ (all)	1.00
4	81	1.00	$d = 63$ (all)	1.00

Table 8: Depth sweep ($n = 64$, path-union). The exact-reach threshold (pairwise ≥ 0.99) sits at *exactly* 3^L for $L = 1, 2$; just past it reach degrades to a partial floor (~ 0.8 – 0.9), not zero. $L \geq 3$ resolves every distance present and is exact.

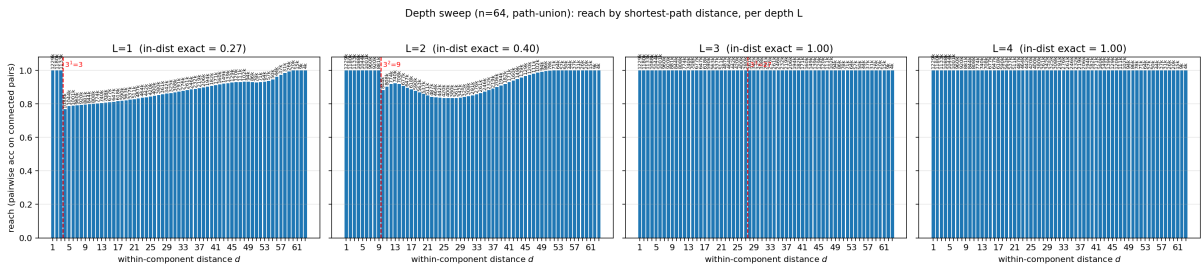


Figure 9: Pairwise accuracy on within-component pairs vs. shortest-path distance d , one panel per depth L (test-pair counts above the bars; the red line marks 3^L). Reach is flat at 1.0 up to 3^L , drops sharply at 3^L+1 for $L = 1, 2$, and stays near-perfect across the whole range for $L = 3, 4$.

What this shows.

- **A sharp $\times 3$ -per-layer threshold.** The exact-reach distance is 3 for $L = 1$ and 9 for $L = 2$ — precisely 3^L , and the transition is sharp ($L = 2$: reach = 1.00 at $d = 9$, 0.88 at $d = 10$). This is the direct, causal explanation of the distance-9 wall throughout Part II: all our $n = 40$ models (and the paper’s main ones) are $L = 2$, so $3^2 = 9$.

- **Beyond the threshold the model degrades gracefully, not to zero.** Past 3^L reach settles at ~ 0.8 – 0.9 (a weak long-range signal, since path-union training does expose distant connected pairs), but ≥ 0.99 is needed to get a whole graph right, so exact match collapses (0.27, 0.40).
- **The wall is a depth limit, and $L = 2$ is genuinely capacity-saturated.** Adding a layer removes it ($L = 3$ is exact at every distance); and $L = 2$ does not improve with training — over the full 1M steps its loss plateaus at ≈ 0.13 with $d^* = 9$, the signature of a capacity floor rather than under-training.

Caveat (the high- L behaviour). The clean $\times 3$ reading is exact at $L = 1, 2$; at $L = 3$ the model already resolves the full $n = 64$ range, which is *beyond* $3^3 = 27$. Two points. First, $n = 64$ caps the largest measurable distance at 63, so we cannot probe past it without a larger n . Second, the paper’s 3^L bound (Theorem 4.5) is *worst-case existential* — it guarantees that *some* diameter- (3^L+1) graph defeats the model, not that *every* one does — and path graphs are very low-degree, evidently not that adversarial case. So this experiment cleanly fixes the $L = 2$ wall at $3^2 = 9$ and shows it lifts with depth; locating the exact capacity of $L \geq 3$ would require adversarial graphs at $n > 64$ (a clean follow-up).

8 Takeaway and outlook

Two points close the report. First, the training-dynamics-within-capacity mechanism — the data lever of Part I — is *not* the fix for this limitation: the wall is a property of the model’s capacity (3^L), and steering the training distribution does not move it. Second, the capacity problem is clearly present in our experiments but its shape is not yet fully understood — in particular, beyond 3^L+1 the accuracy does *not* decay monotonically; it partially recovers and even *rises* at larger distances (e.g. for $L = 1$, reach climbs from 0.79 at $d = 9$ to 0.91 at $d = 40$), which a clean 3^L cut-off does not predict and which we still need to explain. We are currently testing whether a spectral / Laplacian inductive bias (the auxiliary $\text{Tr}(H^\top LH)$ loss and a similarity-based read-out) helps the model improve *beyond* its matrix-powering capacity.